

Classroom Engine

1. Feature Name

Classroom Engine

2. Purpose

- **Problem Solved:** Instructors need a way to group students into a managed cohort tied to a specific learning path, so they can run assignments, track members, and control participation.
 - **Why It Exists:** A classroom provides the membership context used by the Assignment & Submission workflow. Without it, there is no scoped group of students for an instructor to manage.
 - **Role in Application:** It is the social/organizational shell around a learning path. It owns membership (who is in the class, in which role), invite-based joining, member moderation (removing/invalidating students), and hosts assignments.
-

3. What the User Can Do

Instructor

- Create a classroom bound to a learning path (auto-joins as `Instructor`).
- Edit and delete their own classrooms.
- Generate and share the classroom invite code.
- View the member list.
- Remove student members.
- Mark a student member as `Invalid` with a reason (blocks that user's account).

Student

- Join a classroom using an invite code (joins as `Student`).
- View classrooms they belong to.
- Leave a classroom (students only).
- View members and assignments.

Admin

- Same create/edit/delete capabilities as an instructor for classrooms they own; plus admin-level classroom management (list/reassign learning path/detail) exposed through the Admin Console feature.
-

4. Feature Workflow

```
[Instructor creates classroom]
→ POST /api/v1/classrooms { title, description, learningPathId }
→ ClassroomService.CreateAsync
→ Learning path existence checked
→ Classroom row created with InviteCode = 8-char GUID fragment
→ Creator auto-joins as Instructor (UserClassrooms row)
→ CLASSROOM_CREATED audit logged

[Student joins]
→ POST /api/v1/classrooms/join { inviteCode }
→ ClassroomService.JoinAsync → looks up classroom by invite code
→ If already member → "You are already a member."
→ UserClassrooms row created with Role = "Student"
→ CLASSROOM_JOINED audit logged

[Instructor moderates]
→ DELETE /classrooms/{id}/members/{userId} (remove member)
→ POST /classrooms/{id}/members/{userId}/invalid { reason } (mark invalid)
→ Both require the caller to be an Instructor of the classroom (EnsureInstructorAsync)
```

5. How It Works Internally

Invite Code Generation

`ClassroomService.GenerateInviteCode(): Guid.NewGuid().ToString("N")[..8].ToUpper()` — takes the first 8 hex characters of a GUID (no dashes) and uppercases them. The code is stored on the classroom and used to look up memberships on join.

Role Model

Membership roles are stored as a plain string on `UserClassroom.Role` ("Instructor" or "Student"), not as ASP.NET Identity roles. This is a per-classroom role that controls in-classroom actions via helper checks:

- `EnsureInstructorAsync(classroomId, userId)` — the caller must have a `UserClassrooms` row with `Role == "Instructor"`.
- `EnsureAdminAsync(classroomId, userId)` — the caller must be a member AND have the platform `Admin` Identity role (looked up through `Roles / UserRoles` tables).

Ownership Enforcement

`GetOwnedClassroomAsync(id, userId)` loads the classroom and throws `UnauthorizedAccessException("You do not own this classroom.")` when `classroom.CreatedById != userId`. Used by Update and Delete.

Join / Leave Rules

- Join: invite code must match an existing classroom; duplicate membership rejected.
- Leave: an `Instructor`-role member **cannot** leave ("Instructors cannot leave. Transfer ownership first.") — only students may leave.

Member Removal

`RemoveMemberAsync` requires the caller to be an instructor of that classroom and blocks removing other instructors (`"Cannot remove an instructor."`).

Mark Invalid

`MarkInvalidAsync(classroomId, memberUserId, dto, userId)`:

1. Requires the caller to be an instructor of the classroom.
2. Requires a non-empty `Reason`.
3. Only students can be marked invalid (`membership.Role != "Student"` rejected).
4. The Super Admin account cannot be marked invalid.
5. Sets the user's `Status = UserStatus.Invalid` and `InvalidReason = reason` on the **user** entity (this is a platform-wide account state change, not just classroom membership).
6. Logs `USER_MARKED_INVALID` audit with old/new values.

Detail Assembly

`GetByIdAsync(id, userId)` requires the caller to be a member (`UserClassrooms` row for that classroom), then returns:

- Classroom metadata + learning path title.
- Member list (with user `Status` and `InvalidReason`).
- Assignments each annotated with the caller's own submission info (`HasSubmitted`, `MySubmissionId`, `MySubmissionStatus`, `MyGrade`, `MyFeedback`).

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/classroom/ClassroomPage.tsx</code>	List classrooms, create classroom modal, join by invite code
Page	<code>frontend/src/pages/classroom/ClassroomDetailPage.tsx</code>	Members, assignments, invite code, mark invalid/remove member
Component	<code>frontend/src/components/classroom/ClassroomCard.tsx</code>	Classroom card with role-based gradient
Service	<code>frontend/src/services/classroomService.ts</code>	Classroom CRUD, join/leave, member ops, assignment/submission calls
State	<code>frontend/src/redux/slices/classroomSlice.ts</code>	Classroom Redux state
Selectors	<code>frontend/src/redux/selectors/classroomSelectors.ts</code>	Classroom selectors

7. Backend Implementation

Layer	File Path	Responsibility
Controller	backend/Controllers/ClassroomController.cs	Classroom CRUD, join/leave, members, mark invalid (plus assignment/submission endpoints)
Service	backend/Services/Classroom/ClassroomService.cs	Membership logic, invite code, moderation, ownership checks
DTO	backend/DTOs/Classroom/ClassroomDto.cs	Classroom/response/member/admin DTOs
Validator	backend/Validators/Classroom/CreateClassroomValidator.cs	Create-classroom validation
Interface	backend/Interfaces/Services/IClassroomService.cs	Service contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
Classrooms	Classroom metadata (title, description, invite code)	FK <code>LearningPathId</code> → <code>LearningPaths</code> ; FK <code>CreatedById</code> → <code>Users</code> ; 1→N <code>UserClassrooms</code> , 1→N <code>Assignments</code>
UserClassrooms	Membership (many-to-many join with role)	Composite PK (<code>UserId</code> , <code>ClassroomId</code>); FKs → <code>Users</code> and <code>Classrooms</code> ; <code>Role</code> string, <code>JoinedAt</code>

- Membership is a many-to-many relationship between `Users` and `Classrooms` with an explicit `Role` attribute.
- One classroom is bound to exactly one learning path.

9. Security & Authorization

- All classroom endpoints require authentication (`[Authorize]`).
- Viewing a classroom requires membership (`UnauthorizedAccessException("You are not a member of this classroom.")`).
- Update/delete require ownership (`CreatedById == userId`).
- Remove-member, mark-invalid, and assignment operations require the caller to have the per-classroom `Instructor` role (`EnsureInstructorAsync`).
- Creating assignments additionally requires the platform `Admin` Identity role (`EnsureAdminAsync`).
- The Super Admin account is protected from being marked invalid.

10. Important Business Rules

1. **Invite code:** 8-character uppercase code derived from a GUID; join requires an exact match.
2. **Creator auto-join:** The creator is automatically a member with the `Instructor` role.
3. **Instructors cannot leave** a classroom (no ownership transfer implemented).
4. **Cannot remove an instructor** as a member.
5. **Only students can be marked invalid**, and a reason is required; the Super Admin is exempt.
6. **Duplicate join** is rejected.
7. **Deletion** removes the classroom row (assignment/submission cleanup handled by FK configuration).

11. Example of Internal Execution

Step 1: Instructor creates a classroom

- **User Action:** Instructor fills the create form (title, description, learning path).
- **Frontend:** `classroomService.create` → `POST /api/v1/classrooms`.
- **Service:** `CreateAsync` verifies the learning path exists, creates a `Classrooms` row with invite code (e.g. `3F2A9C1B`), and inserts the creator as an `Instructor` membership.
- **Response:** `ClassroomResponseDto`; the frontend lists the classroom and shows the invite code.

Step 2: Student joins

- **User Action:** Student enters invite code `3F2A9C1B`.
- **Service:** `JoinAsync` finds the classroom, rejects if already a member, inserts a `Student` membership row.
- **Response:** success; the classroom now appears in the student's list with `UserRole = "Student"`.

Step 3: Instructor marks a student invalid

- **User Action:** Instructor selects a student and provides a reason.
- **Service:** `MarkInvalidAsync` → instructor role check passes → student role verified → user's `Status` set to `Invalid` with `InvalidReason`.
- **Effect:** The student's account is now `Invalid` platform-wide and cannot log in (per the Authentication feature).

12. Files Involved

Frontend

- `frontend/src/pages/classroom/ClassroomPage.tsx`
- `frontend/src/pages/classroom/ClassroomDetailPage.tsx`
- `frontend/src/components/classroom/ClassroomCard.tsx`
- `frontend/src/services/classroomService.ts`
- `frontend/src/redux/slices/classroomSlice.ts`
- `frontend/src/redux/selectors/classroomSelectors.ts`

Backend

- `backend/Controllers/ClassroomController.cs`
- `backend/Services/Classroom/ClassroomService.cs`
- `backend/DTOs/Classroom/ClassroomDto.cs`
- `backend/Validators/Classroom/CreateClassroomValidator.cs`
- `backend/Interfaces/Services/IClassroomService.cs`

Database

- `backend/Entities/Classroom.cs`
 - `backend/Entities/UserClassroom.cs`
 - `backend/Configurations/ClassroomConfiguration.cs` (EF configuration)
-

13. Functionality

- Create / edit / delete classrooms bound to a learning path
- Generate and share an invite code
- Join a classroom by invite code
- Leave a classroom (students only)
- List members with roles/status
- Remove a member (instructor only)
- Mark a student invalid with a reason (instructor only)
- View per-member submission/assignment info in classroom detail
- Audit logging of create/update/join/leave/remove/mark-invalid